

PERANCANGAN SISTEM TERDISTRIBUSI LAYANAN SERVICE HP TOKO DIGIKAR



Oleh:
SYAWAL ICHWANUL RAFKI
23010220013

FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS MUHAMMADIYAH BANTEN

2026

PERANCANGAN SISTEM TERDISTRIBUSI LAYANAN SERVICE HP TOKO DIGIKAR

Dokumentasi Proyek dan Implementasi Kode Microservices Berbasis Node.js

Mata Kuliah: Sistem Terdistribusi

Dosen Pengampu: Ahmad Roihan, M.TI.

Format Laporan: Buku Panduan Per Bab (UTS)

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS MUHAMMADIYAH BANTEN

2026

BAB I: Pemilihan Model Arsitektur Terdistribusi

Dalam merancang sistem informasi manajemen bengkel perbaikan kustom seperti **Toko Digikar**, pemilihan model arsitektur terdistribusi menjadi fondasi penentu keandalan sistem. Berdasarkan analisis kebutuhan operasional yang mencakup pendaftaran jadwal konsumen, manajemen antrean secara dinamis (real-time), dan pengiriman notifikasi otomatis, model arsitektur terdistribusi yang paling sesuai untuk diterapkan adalah **Arsitektur Microservices**.

Berbeda dengan arsitektur monolitik konvensional yang menyatukan seluruh fungsi bisnis ke dalam satu basis kode dan satu proses eksekusi tunggal, Arsitektur Microservices memecah sistem menjadi sekumpulan layanan kecil mandiri. Setiap layanan (service) bertanggung jawab penuh atas satu domain kemampuan bisnis tertentu, berjalan di prosesnya masing-masing, memiliki basis datanya sendiri secara logis, dan berkomunikasi melalui protokol jaringan standar yang ringan seperti HTTP/REST API.

BAB II: Analisis Pertimbangan dan Justifikasi Teknikal

Penentuan opsi Arsitektur Microservices didasarkan pada tiga parameter utama analisis terdistribusi yang selaras dengan permasalahan di lapangan:

teknikal

A. Karakteristik Beban Kerja (I/O-Bound vs CPU-Bound)

Aktivitas utama dari operasional sistem Toko Digikar didominasi oleh operasi **I/O-bound**. Proses seperti pengisian formulir pendaftaran jadwal kedatangan, pengecekan nomor urutan antrean oleh konsumen, pembaruan status pengerjaan perangkat oleh teknisi, serta penembakan notifikasi keluar (melalui SMS atau WhatsApp Gateway) sangat bergantung pada kecepatan manipulasi data pada storage (baca/tulis database) serta latensi komunikasi jaringan.

Sistem ini tidak memerlukan operasi komputasi matematika kompleks yang intensif menggunakan prosesor (CPU-bound). Melalui microservices, komponen yang menangani operasi I/O-bound intensif ini dapat diisolasi dan dioptimalkan secara mandiri tanpa membebani thread proses utama aplikasi.

B. Kebutuhan Skalabilitas Horizontal

Toko Digikar memiliki potensi fluktuasi beban pengguna yang sangat dinamis, misalnya lonjakan trafik pendaftaran jadwal di akhir pekan atau saat masa promo kupon service. Dalam arsitektur microservices, jika komponen pendaftaran atau antrean mengalami bottleneck akibat ribuan akses konsumen secara simultan, tim teknis dapat melakukan **skalabilitas horizontal (scale-out)** secara spesifik.

Skalabilitas ini diwujudkan dengan menambah replika instans server untuk *Service Antrean* saja di balik penyeimbang beban (load balancer), tanpa perlu menduplikasi komponen sistem lain seperti *Service Notifikasi* yang beban kerjanya masih stabil. Pendekatan ini jauh lebih efisien dalam penggunaan resource resource komputasi.

C. Tingkat Kemandirian Antar Komponen Sistem

Kemandirian komponen diwujudkan melalui prinsip dampak isolasi kegagalan (*fault isolation*). Logika bisnis antrean dan notifikasi dipisahkan secara fisik. Apabila penyedia pihak ketiga penembak WhatsApp Gateway mengalami crash atau kehabisan kuota, kegagalan tersebut hanya akan menghentikan *Service Notifikasi* secara lokal.

Layanan utama seperti API Gateway dan *Service Antrean* tetap dapat berjalan 100% normal. Konsumen tetap bisa datang ke toko, mengambil nomor antrean, dan teknisi tetap bisa memperbarui database internal pengerjaan HP. Data notifikasi yang gagal dikirim dapat ditahan sementara untuk kemudian diproses ulang begitu layanan notifikasi pulih.

BAB III: Perancangan Mekanisme Komunikasi Komponen

Arsitektur terdistribusi yang dirancang untuk Toko Digikar membagi sistem ke dalam komponen-komponen independen dengan memanfaatkan konsep **API Gateway** sebagai fasilitator komunikasi luar (client) ke dalam sistem internal (backend microservices). Berikut adalah tabel perincian tugas dan penempatan port jaringan lokal dari komponen sistem terdistribusi:

Nama Komponen	Port Lokal	Tugas Utama & Tanggung Jawab Operasional
API Gateway	3000	Menjadi pintu masuk tunggal (Single Entry Point) bagi seluruh client (aplikasi web/mobile konsumen dan dashboard teknisi). Bertanggung jawab melakukan routing request, validasi token keamanan, serta enkapsulasi arsitektur internal.
Service Antrean	3001	Mengelola seluruh domain logika bisnis pendaftaran jadwal konsumen, pencatatan kerusakan HP, penomoran urutan antrean lokal, serta manipulasi database status perbaikan HP.
Service Notifikasi	3002	Bertindak sebagai sub-sistem yang menerima instruksi dari Service Antrean untuk menyusun string pesan teks pengingat dan menembakkannya ke nomor WhatsApp tujuan konsumen.

Prinsip Loose Coupling: Setiap service tidak diizinkan untuk melakukan kueri secara langsung ke database milik service lainnya. Pertukaran data murni dilakukan dengan mengirimkan muatan (payload) terstruktur berformat JSON melalui protokol komunikasi antar-proses resmi yang disepakati.

BAB IV: Penentuan Protokol Komunikasi Data

Untuk menyeimbangkan kebutuhan respon sistem yang instan dan ketahanan sistem terhadap kegagalan jaringan, rancangan ini menggabungkan penggunaan jenis protokol komunikasi sinkron langsung dengan manajemen modular internal:

A. Protokol Sinkron (REST API - HTTP/JSON)

Digunakan pada aktivitas **Pendaftaran Jadwal oleh Konsumen**. Ketika konsumen memilih waktu kedatangan untuk service HP, aplikasi client harus mengirimkan permintaan melalui metode HTTP POST secara sinkron langsung ke API Gateway yang diteruskan ke Service Antrean. Konsumen wajib menunggu di tempat (blocking) hingga server memproses kueri dan mengembalikan respon sukses berstatus HTTP 201 Created yang berisi nomor kode antrean unik. Hal ini penting untuk menghindari alokasi jadwal ganda (double-booking).

B. Protokol Komunikasi Antar-Service Direct

Digunakan pada saat **Teknisi Menyelesaikan Perbaikan HP**. Ketika teknisi menekan tombol "Selesai", Service Antrean memperbarui database internalnya, kemudian secara asinkronus (dari sudut pandang kenyamanan teknisi) mengirimkan sinyal data ke Service Notifikasi melalui HTTP POST internal pada port 3002. Dengan metode pemisahan delegasi ini, dashboard teknisi tidak akan mengalami lag atau freeze karena beban pengiriman pesan teks telah dialihkan sepenuhnya ke modul notifikasi mandiri.

BAB V: Cetak Biru Implementasi Kode Program (VS Code)

Berikut adalah baris kode utuh siap pakai yang diimplementasikan langsung pada aplikasi Visual Studio Code menggunakan lingkungan runtime Node.js dan framework minimalis Express.

1. API Gateway (gateway.js Port: 3000)


```

const express = require('express');
const bodyParser = require('body-parser');
const http = require('http');

const app = express();
app.use(bodyParser.json());
const PORT = 3000;

app.post('/api/daftar', (req, res) => {
  const data = JSON.stringify(req.body);
  const options = {
    hostname: 'localhost', port: 3001, path: '/daftar', method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Content-Length':
data.length }
  };
  const request = http.request(options, (response) => {
    let body = "";
    response.on('data', chunk => body += chunk);
    response.on('end', () =>
res.status(response.statusCode).send(JSON.parse(body)));
  });
  request.on('error', () => res.status(500).json({ error: "Service Antrean
Offline" }));
  request.write(data); request.end();
});

app.post('/api/service-selesai', (req, res) => {
  const data = JSON.stringify(req.body);
  const options = {
    hostname: 'localhost', port: 3001, path: '/selesai', method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Content-Length':
data.length }
  };
  const request = http.request(options, (response) => {
    let body = "";
    response.on('data', chunk => body += chunk);
    response.on('end', () =>
res.status(response.statusCode).send(JSON.parse(body)));
  });
  request.on('error', () => res.status(500).json({ error: "Service Antrean
Offline" }));
  request.write(data); request.end();
});

app.listen(PORT, () => console.log(` [API GATEWAY] aktif di port ${PORT} `));

```

2. Service Antrean & Pendaftaran (service-antrean.js Port: 3001)

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();
app.use(bodyParser.json());
const PORT = 3001;

app.post('/daftar', (req, res) => {
  const { nama, tipeHP } = req.body;
  if (!nama || !tipeHP) return res.status(400).json({ status: "Gagal", message: "Data kosong" });
  const noAntrean = "DGK-" + Math.floor(1000 + Math.random() * 9000);
  console.log(` [ANTREAN] Sukses Mendaftar: ${nama} [${noAntrean}]`);
  res.status(201).json({ status: "Sukses", data: { noAntrean, nama, tipeHP } });
});

app.post('/selesai', async (req, res) => {
  const { noAntrean, namaKonsumen, noWa } = req.body;
  console.log(` [ANTREAN] HP ${noAntrean} Berhasil Diperbaiki. Mengirim data ke Notifikasi...`);
  try {
    const response = await fetch('http://localhost:3002/kirim-notif', {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        noAntrean, nama: namaKonsumen, telepon: noWa,
        pesan: `Halo ${namaKonsumen}, HP Anda (${noAntrean}) di Toko Digikar SUDAH SELESAI di-service.`
      })
    });
    const hasilNotif = await response.json();
    res.status(200).json({ status: "Sukses", detailNotif: hasilNotif });
  } catch (error) {
    res.status(500).json({ status: "Gagal", error: "Service Notifikasi mati" });
  }
});

app.listen(PORT, () => console.log(` [SERVICE ANTREAN] aktif di port ${PORT}`));
```

3. Service Notifikasi (service-notif.js - Port: 3002)

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();
app.use(bodyParser.json());
const PORT = 3002;

app.post('/ kirim-notif', (req, res) => {
  const dataNotif = req.body;
  console.log(`
  -----`);
  console.log(` [NOTIFIKASI SMS/WHATSAPP GATEWAY OUTBOUND]`);
  console.log(` Nomor Tujuan : ${dataNotif.telepon}`);
  console.log(` Teks Pesan   : ${dataNotif.pesan}`);
  console.log(` -----`);
  res.status(200).json({ status: "Ter kirim", timestamp: new
Date().toISOString() });
});

app.listen(PORT, () => console.log(` [SERVICE NOTIFIKASI] aktif di port ${PORT}`));
```

BAB VI: Mekanisme Urutan Logis dan Replikasi Data

A. Sinkronisasi Urutan Logis (Logical Clock)

Dalam sistem terdistribusi, waktu fisik antar-mesin server tidak selalu presisi secara milidetik (efek clock skew). Guna mengurutkan kedatangan konsumen secara adil tanpa bergantung pada jam fisik perangkat, sistem ini menggunakan implementasi *Lamport Logical Clock* yang disederhanakan ke dalam bentuk generator nomor urut atomik sekuensial format string DGK-XXXX.

Setiap transaksi pendaftaran baru memicu inkremen nilai counter $L = L + 1$ pada basis data terpusat, menjamin urutan antrean mutlak konsisten berdasarkan waktu logis kejadian request tiba di *Service Antrean*.

B. Strategi Replikasi Data dan Ketahanan (Fault Tolerance)

Untuk menghindari malapetaka kehilangan data akibat server padam (Single Point of Failure), database milik *Service Antrean* dikonfigurasi dengan skema **Master-Slave Replication**. Server Master yang bertempat di server cloud pusat menangani penuh operasi penulisan data pendaftaran baru.

Setiap data baru yang masuk akan langsung didistribusikan secara asinkron ke server cadangan (Slave) yang ditempatkan secara fisik pada komputer lokal Toko Digikar. Jika koneksi internet publik menuju cloud terputus, server lokal toko dapat langsung dialihkan (failover) menjadi server baca darurat sehingga operasional harian pengambilan HP di kasir toko tidak akan lumpuh.

Diagram alur komunikasi

